Optimized Parallel Matrix Multiplication

A Project Component for

Parallel and Distributed Computing (UCS645)

By

Sr	Name	Roll No
1	Aniket Gupta	102317169

Under the guidance of

Dr. Saif Nalband

(Assistant Professor, DCSE)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(DEEMED TO BE UNIVERSITY)

PATIALA - 147004

APRIL, 2026

Table of Contents

1 Introduction **1**

1.1 Background 1

1.2 Introduction to Problem Statement 1

2 Literature Review **2**

3 Research Gaps **3**

4 Problem Formulation **4**

5 Objectives **4**

6 Methodology **5**

6.1 Algorithm 5

6.2 Equation 5

7 Datasets **7**

8 Results and Discussion **8**

References **9**

List of Figures

1 Parallel Matrix Multiplication using OpenMP 5

List of Tables

1 Summary of Literature Review 2
2 Performance Comparison 8

1 Introduction

Parallel computing techniques such as OpenMP enable efficient utilization of multi-core processors [1].

1.1 Background

Matrix multiplication is a fundamental operation used in various domains such as scientific computing, machine learning, image processing, and numerical simulations. The traditional sequential matrix multiplication algorithm has a time complexity of $O(n^3)$, which makes it computationally expensive for large matrices.

With the advancement of multi-core processors, parallel computing has emerged as an effective solution to reduce execution time. OpenMP is a widely used API that enables parallel programming on shared-memory architectures.

1.2 Introduction to Problem Statement

Despite the availability of powerful processors, inefficient utilization of computational resources can lead to performance bottlenecks. Sequential execution fails to exploit multiple cores, resulting in longer computation time.

This project focuses on optimizing matrix multiplication using parallel computing techniques such as OpenMP, cache blocking, and scheduling strategies to improve performance and scalability.

2 Literature Review

Parallel programming models have been extensively studied in literature [2].

Table 1: Summary of Literature Review

References	**Concepts Highlighted**	**Technique(s) used**	**Significance**	**Limitations**
OpenMP Studies	Parallel Computing	OpenMP APIs	Improved execution time using multi-core processors	Limited to shared memory systems
HPC Research	Matrix Multiplication Optimization	Cache Blocking	Improved cache utilization	Performance depends on block size
Scheduling Research	Load Balancing	Dynamic and Guided Scheduling	Better distribution of workload	Overhead in scheduling

3 Research Gaps

- Existing sequential approaches do not utilize multi-core processors effectively.
- Cache utilization is often inefficient in basic implementations.
- Static scheduling may lead to load imbalance across threads.

4 Problem Formulation

Matrix multiplication involves computing the product of two matrices, which requires a large number of arithmetic operations. The traditional approach suffers from high computational complexity and inefficient memory usage.

The problem is to design an optimized matrix multiplication algorithm that reduces execution time by leveraging parallel processing and memory optimization techniques such as cache blocking and dynamic scheduling.

5 Objectives

- To implement sequential matrix multiplication.
- To develop a parallel implementation using OpenMP.
- To apply cache blocking for memory optimization.
- To compare scheduling strategies (static, dynamic, guided).
- To evaluate performance using execution time and speedup.

6 Methodology

Cache blocking is used to improve memory performance by reducing cache misses [3].

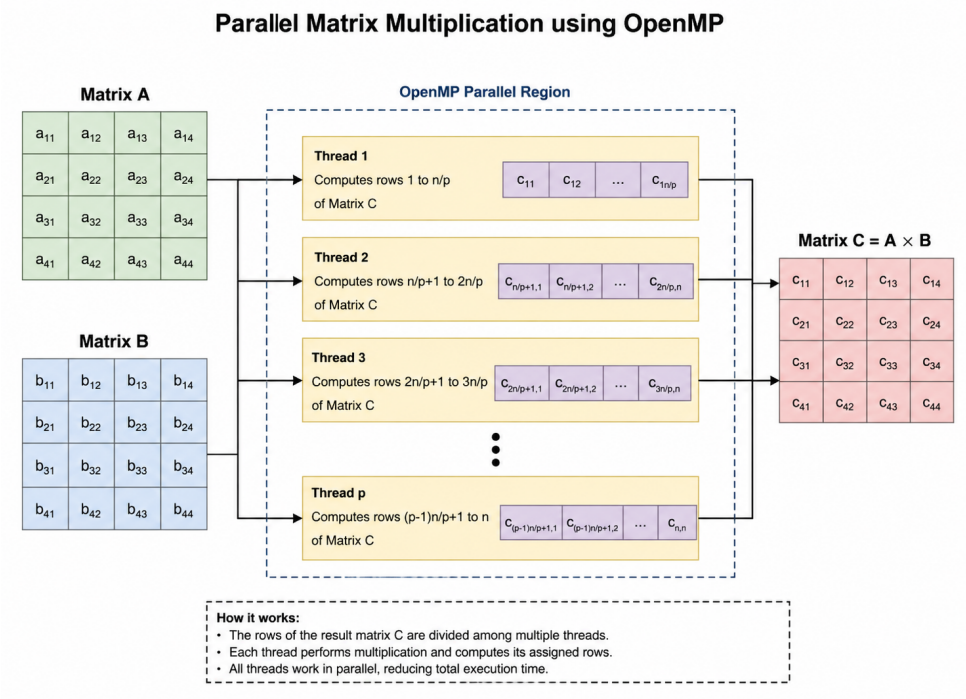

Figure 1: Parallel Matrix Multiplication using OpenMP

- Sequential implementation is used as a baseline.
- OpenMP is used to parallelize matrix multiplication.
- Cache blocking technique is applied to improve memory access.
- Scheduling strategies (static, dynamic, guided) are compared.

6.1 Algorithm

6.2 Equation

The time complexity of matrix multiplication is:

$$O(n^3)$$

Speedup is calculated as:

Algorithm 1 Parallel Matrix Multiplication

```
for i = 1 to n do
    for j = 1 to n do
        for k = 1 to n do
            C[i][j] = C[i][j] + A[i][k] × B[k][j]
        end for
    end for
end for
```

$$Speedup = \frac{T_{sequential}}{T_{parallel}}$$

7 Datasets

This project does not use any external dataset. Random matrices are generated for performance evaluation.

8 Results and Discussion

Method	Time (seconds)	Speedup
Sequential	0.904	1.00
Parallel	0.235	3.85
Static	0.279	3.24
Dynamic	0.124	7.29
Guided	0.112	8.07
Cache Blocking	0.945	0.96

Table 2: Performance Comparison

The results show that parallel execution significantly improves performance. Guided scheduling provides the best results due to efficient load balancing.

Cache blocking did not improve performance in this case, possibly due to overhead or smaller matrix size.

References

[1] OpenMP Architecture Review Board, "Openmp application programming interface," https://www.openmp.org/, 2023.

[2] P. Pacheco, *An Introduction to Parallel Programming.* Morgan Kaufmann, 2011.

[3] V. Authors, "Cache blocking optimization techniques," https://en.wikipedia.org/wiki/Cache_blocking, 2020.